

LiDAR-Based SLAM

Shambhavi Singh

ECE 276A: Sensing and Estimation in Robotics
University of California San Diego

Abstract—This project implements a full SLAM system for a differential-drive robot using encoder and IMU odometry, LiDAR-based scan matching with Iterative Closest Point (ICP), occupancy-grid mapping, RGBD texture mapping, and pose-graph optimization in GTSAM. Encoder and IMU measurements first provide a motion prior, which is then refined through LiDAR scan alignment. To reduce long-term drift, loop closure constraints are incorporated into a factor graph and optimized with nonlinear least squares. The optimized poses are used to build both occupancy maps and textured floor maps of the environment.

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) is a central problem in mobile robotics in which a robot must estimate its motion while building a map of an unknown environment. Accurate localization and mapping are essential for autonomous navigation, especially in indoor settings where GPS is unavailable or unreliable.

Wheel encoder odometry provides frequent motion estimates, but the resulting trajectory accumulates error over time due to wheel slip, calibration mismatch, and sensor noise. In this project, encoder-based motion is combined with IMU yaw-rate measurements to form a motion-model prior. While this prior still drifts over long time horizons, it provides a stable initialization for scan matching and reduces the search space for estimating relative pose between scans.

To mitigate odometry drift, we use a LiDAR front-end that aligns consecutive scans with ICP. Scan matching yields locally consistent relative pose estimates that refine the encoder-IMU trajectory and improve mapping quality. Global consistency is enforced through pose graph optimization using factor graphs. When previously visited locations are detected, loop closure constraints are added, allowing accumulated error to be corrected through nonlinear optimization.

Finally, LiDAR and RGBD measurements are combined to produce two map representations: (i) a 2-D occupancy grid describing free and occupied space and (ii) a textured floor map obtained by projecting RGBD observations into the world frame.

II. PROBLEM FORMULATION

The robot pose at time t is defined as

$$\mathbf{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} \in SE(2). \quad (1)$$

Given encoder measurements $\mathbf{c}_{1:T}$, yaw-rate measurements $\omega_{1:T}$, and LiDAR scans $\mathcal{L}_{1:T}$, the objective is to estimate the trajectory

$$\mathcal{X} = \{\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T\}. \quad (2)$$

The differential-drive motion model is

$$\mathbf{x}_k = \begin{cases} \mathbf{x}_{k-1} + \begin{bmatrix} v\Delta t \cos \theta_{k-1} \\ v\Delta t \sin \theta_{k-1} \\ 0 \end{bmatrix} & |\omega| \approx 0 \\ \mathbf{x}_{k-1} + \begin{bmatrix} \frac{v}{\omega} (\sin(\theta_{k-1} + \omega\Delta t) - \sin \theta_{k-1}) \\ \frac{v}{\omega} (\cos \theta_{k-1} - \cos(\theta_{k-1} + \omega\Delta t)) \\ \omega\Delta t \end{bmatrix} & \text{otherwise.} \end{cases} \quad (3)$$

III. TECHNICAL APPROACH

A. Sensor Suite and Data Notes (Project Specs)

The dataset contains synchronized measurements from four primary sensors:

Encoders (40 Hz). Encoders measure wheel rotation for all four wheels, and the encoder counter is reset after each readout. As a result, the recorded encoder values are already *incremental* tick counts (they should *not* be differenced again). For example, five consecutive counts of 0, 1, 0, -2, 3 correspond to $(0 + 1 + 0 - 2 + 3)\ell = 2\ell$ meters traveled for that wheel over the window. The wheel diameter is 0.254 m and with 360 ticks per revolution, each tick corresponds to 0.0022 m of travel.

IMU. Linear acceleration and angular velocity are provided; in our pipeline we use only the IMU *yaw rate* as the angular velocity ω in the differential-drive model.

Hokuyo LiDAR. A Hokuyo UTM-30LX horizontal LiDAR provides 270° field-of-view scans (1081 ranges per scan) with maximum range 30 m. We convert ranges to (x, y) points in the sensor frame and then transform them into the robot body and world frames using known extrinsics and the estimated pose.

Kinect RGBD. The Kinect provides RGB images and disparity. The depth camera is located at (0.18, 0.005, 0.36) m relative to the robot center and has orientation roll 0 rad, pitch 0.36 rad, yaw 0.021 rad. The depth intrinsics are

$$\mathbf{K} = \begin{bmatrix} 585.05 & 0 & 242.94 \\ 0 & 585.05 & 315.84 \\ 0 & 0 & 1 \end{bmatrix}. \quad (4)$$

Kinect timestamps are irregular (frames may be dropped, with gaps up to ≈ 0.2 s), so each RGBD frame is matched to the

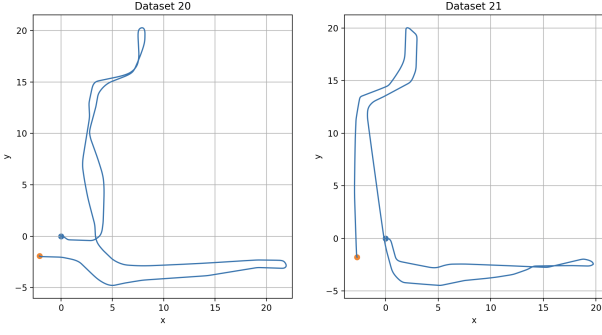


Fig. 1. Encoder+IMU odometry trajectories (Datasets 20 and 21).

nearest available SLAM pose in time. In addition, the RGB and depth cameras are not co-located; we use the provided mapping from disparity to depth and RGB pixel coordinates (detailed in Sec. III-F).

B. Encoder and IMU Odometry

The robot provides encoder measurements at approximately 40 Hz as incremental tick counts

$$\mathbf{c}_t = [c_{FR}, c_{FL}, c_{RR}, c_{RL}], \quad (5)$$

corresponding to front-right, front-left, rear-right, and rear-left wheels. Since the measurements are incremental, they do not require differencing across timestamps. Each encoder tick corresponds to $s = 0.0022$ m of travel.

Right and left wheel distances are

$$d_R = \frac{c_{FR} + c_{RR}}{2} s, \quad (6)$$

$$d_L = \frac{c_{FL} + c_{RL}}{2} s, \quad (7)$$

and the forward displacement is

$$d = \frac{d_R + d_L}{2}. \quad (8)$$

Linear velocity is computed as $v = d/\Delta t$. IMU yaw-rate measurements are synchronized to encoder timestamps using linear interpolation to obtain angular velocity ω . The arc-based motion model described in Section II is used to integrate the pose over time.

1) *Odometry-only trajectory*: Figure 1 shows trajectories estimated using encoder and IMU odometry alone for Datasets 20 and 21.

a) *Observations.*: The odometry-only trajectories in Fig. 1 follow the overall shape of the robot motion and capture both straight corridor segments and turning behavior. However, drift becomes increasingly visible over longer distances. In Dataset 20, the lower corridor gradually shifts laterally, consistent with small heading biases integrating into larger position errors. Dataset 21 shows stronger accumulated deviation toward the end of the run, where the path departs from the expected corridor geometry. This behavior is expected: wheel odometry accumulates bias from slip and calibration

mismatch, and integrating IMU yaw-rate introduces additional bias over time. Despite these limitations, the encoder+IMU trajectory remains physically plausible and provides a strong motion prior for initializing ICP scan matching.

C. ICP Warm-up: 3D Point Cloud Registration

To verify the ICP implementation prior to LiDAR scan matching, we solve a 3D registration task using depth-derived point clouds and canonical object models. Given a model point set $\mathcal{P} = \{\mathbf{p}_i\}$ and an observed point set $\mathcal{Q} = \{\mathbf{q}_j\}$, ICP iterates between:

- 1) **Correspondence**: assign each $\mathbf{p}_i$ to its nearest neighbor in $\mathcal{Q}$ (KD-tree).
- 2) **Alignment**: estimate the rigid transform $(\mathbf{R}, \mathbf{t})$ that minimizes squared error,

$$\min_{\mathbf{R} \in SO(3), \mathbf{t} \in \mathbb{R}^3} \sum_i \|\mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_{\pi(i)}\|_2^2. \quad (9)$$

The alignment update is computed using an SVD/Kabsch solution. Since ICP converges to a local optimum, initialization is important. Following the project guidance, we assume the object rotates primarily about the z axis and discretize candidate yaw angles; for each candidate we run ICP and select the solution with the lowest correspondence mean-square error (MSE).

1) *Warm-up results: Drill and liquid container*: Figures 2 and 3 show the final registration overlays for four viewpoints of each object. The canonical model points (red) align closely with the observed point clouds (blue), indicating successful pose estimation across views.

a) *Observations.*: Across the drill views (Fig. 2), the model matches the dominant object geometry: the cylindrical body and major surfaces overlap well with the observed point cloud, suggesting that the rigid transform is recovered correctly. Small residual errors are most apparent near thin structures (e.g., the handle or sharp edges) where depth measurements are sparse and self-occlusion is significant. For the liquid container (Fig. 3), registration is strong in most views, and the large curved surface yields stable correspondences. In some views, overlap decreases on the back side due to missing depth points, slightly increasing correspondence error. Overall, these results validate (i) KD-tree nearest-neighbor matching and (ii) SVD/Kabsch alignment updates, and they reinforce that good initialization is important because poor yaw initialization can lead to local-minimum solutions.

D. LiDAR Scan Matching via 2D ICP

After forming a motion prior from encoder+IMU odometry and validating ICP in the warm-up, we apply LiDAR scan matching to refine the robot trajectory. Given two consecutive Hokuyo scans at times t and $t+1$, we estimate the relative 2D transform ${}^t\mathbf{T}_{t+1} \in SE(2)$ that aligns scan $t+1$ to scan t .

a) *Scan preprocessing and extrinsics.*: Each scan contains 1081 ranges over a 270° field of view (Hokuyo UTM-30LX). Valid measurements are converted to Cartesian points in the LiDAR frame, $\mathbf{p}_i^\ell = [r_i \cos \phi_i, r_i \sin \phi_i]^T$, and points

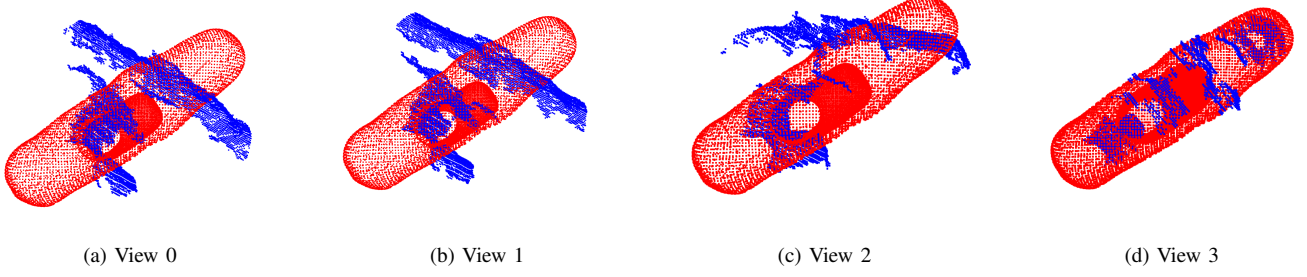


Fig. 2. ICP warm-up registration results for the **drill** across four viewpoints. Red: canonical model. Blue: observed depth point cloud.

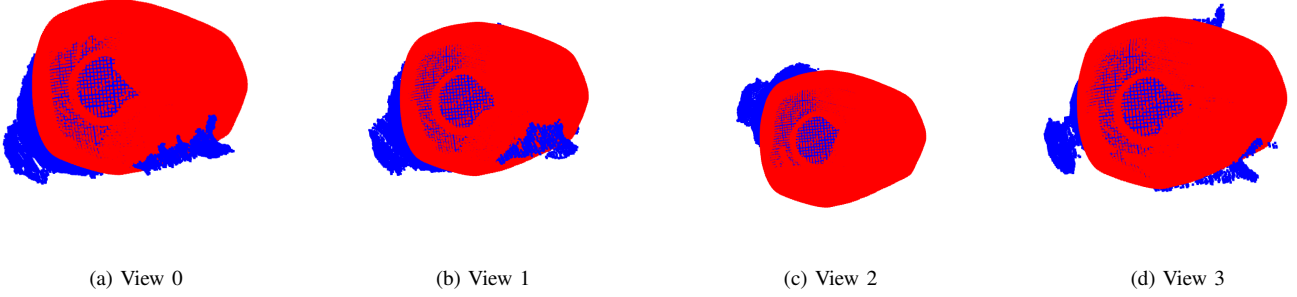


Fig. 3. ICP warm-up registration results for the **liquid container** across four viewpoints. Red: canonical model. Blue: observed depth point cloud.

with $r_i \notin [0.15, 30]$ m are discarded. LiDAR points are transformed to the body frame using known extrinsics. With the body-frame origin at the rear axle, the LiDAR is translated forward by 0.30183 m along the body x -axis, i.e., ${}^b\mathbf{t}_\ell = [0.30183, 0]^T$.

b) Odometry initialization.: Because ICP converges to a local optimum, the initial guess has a strong effect on performance. We initialize the scan-to-scan transform using the encoder+IMU motion prior:

$${}^t\mathbf{T}_{t+1}^{(0)} = \begin{bmatrix} \mathbf{R}(\Delta\theta_t^{\text{odom}}) & \mathbf{t}_t^{\text{odom}} \\ \mathbf{0}^T & 1 \end{bmatrix}. \quad (10)$$

c) ICP optimization (2D).: We run planar ICP (yaw-only rotation and 2D translation). Each iteration: (i) transforms source points using the current estimate, (ii) computes correspondences via nearest-neighbor search (KD-tree), and (iii) updates $(\Delta\mathbf{R}, \Delta\mathbf{t})$ using an SVD/Kabsch alignment in $\mathbb{R}^2$ to minimize the point-to-point objective

$$\min_{\mathbf{R} \in SO(2), \mathbf{t} \in \mathbb{R}^2} \sum_i \|\mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_{\pi(i)}\|_2^2. \quad (11)$$

The iterations stop when the change in mean-square correspondence error falls below a threshold or when the maximum number of iterations is reached.

d) Pose composition.: After estimating the relative motion ${}^t\mathbf{T}_{t+1}$, we recover global poses by composition:

$$\mathbf{T}_{t+1} = \mathbf{T}_t {}^t\mathbf{T}_{t+1}. \quad (12)$$

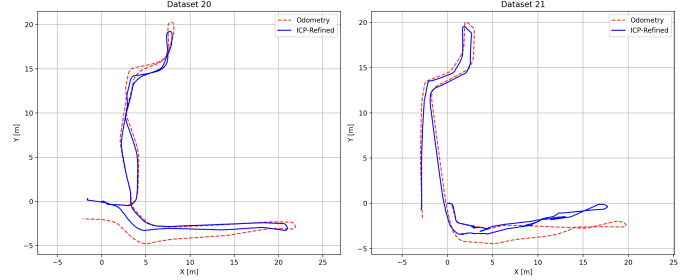


Fig. 4. Trajectory comparison for Datasets 20 and 21. Red dashed: encoder+IMU odometry. Blue: ICP-refined trajectory.

e) Scan matching results.: Figure 4 compares encoder+IMU odometry (red dashed) with the ICP-refined trajectory (blue). By reducing local drift and improving geometric consistency, scan matching directly improves mapping quality.

f) Observations.: Relative to the odometry-only estimate, the ICP-refined path in Fig. 4 is more self-consistent, particularly in long corridor segments. In Dataset 20, scan matching reduces lateral drift and excessive curvature in the lower corridor and improves alignment around the upper loop. In Dataset 21, ICP corrects a substantial portion of drift along the long horizontal corridor, producing a trajectory that better matches a single coherent corridor geometry rather than two diverging tracks. These results suggest that scan-to-scan registration provides additional constraints beyond dead reckoning. Remaining drift is expected because scan matching is local and cannot fully correct global error without loop

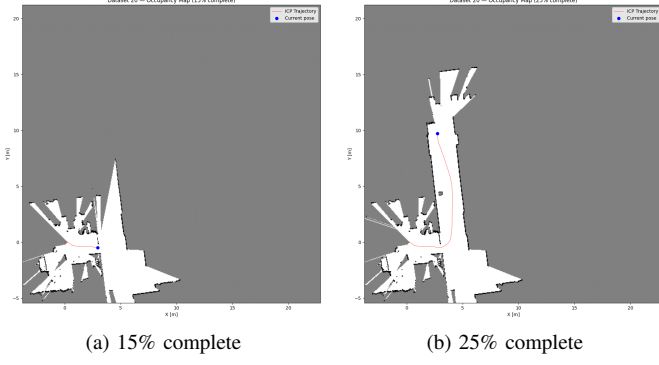


Fig. 5. Dataset 20 occupancy grid map progression. Red: ICP trajectory. Blue: current pose.

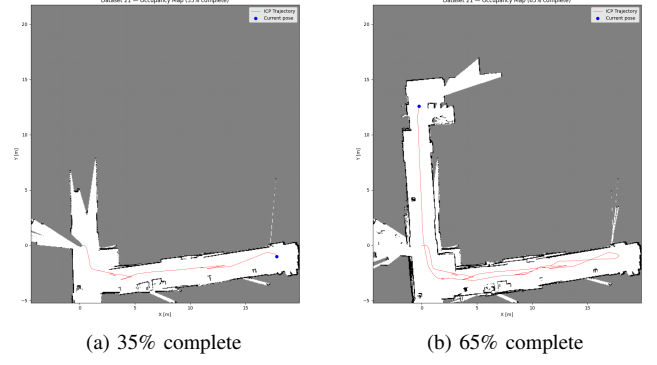


Fig. 6. Dataset 21 occupancy grid map progression. Red: ICP trajectory. Blue: current pose.

closure constraints.

E. Occupancy Grid Mapping

A 2-D occupancy grid map is built using LiDAR scans and the estimated trajectory. Each grid cell stores a log-odds occupancy value

$$L = \log \frac{p}{1-p}. \quad (13)$$

For each LiDAR ray, cells along the ray are updated as free space and the endpoint cell is updated as occupied using Bresenham ray tracing. Log-odds values are clamped to prevent divergence.

1) *Occupancy map progression over time:* To satisfy the requirement of visualizing occupancy grid evolution, we show intermediate maps at fixed completion percentages for each dataset.

a) *Dataset 20.:*

b) *Observations.:* At 15% completion (Fig. 5), the map shows only the local area near the start, and the dominant effect is free-space carving along LiDAR rays. By 25% completion, the main corridor structure becomes clearly visible and walls appear sharper and more continuous. This indicates that scan-to-scan alignment remains stable enough to accumulate many scans without severe wall duplication. Minor speckle noise and small gaps are expected due to range dropouts and grid discretization.

c) *Dataset 21.:*

d) *Observations.:* At 35% completion (Fig. 6), the main corridor is already visible, but turning regions show some wall thickening and small alignment errors. By 65% completion, the large corridor segment is mapped more coherently and the occupancy boundary becomes cleaner, suggesting that ICP has reduced most local drift. Remaining artifacts (e.g., slightly thick walls) are consistent with residual global drift that is typically corrected only after loop closure and pose-graph optimization.

2) *Final occupancy map results:* Figures 7 and 8 show occupancy maps constructed using the ICP-refined trajectory, where corridor walls and room boundaries appear sharp and internally consistent.

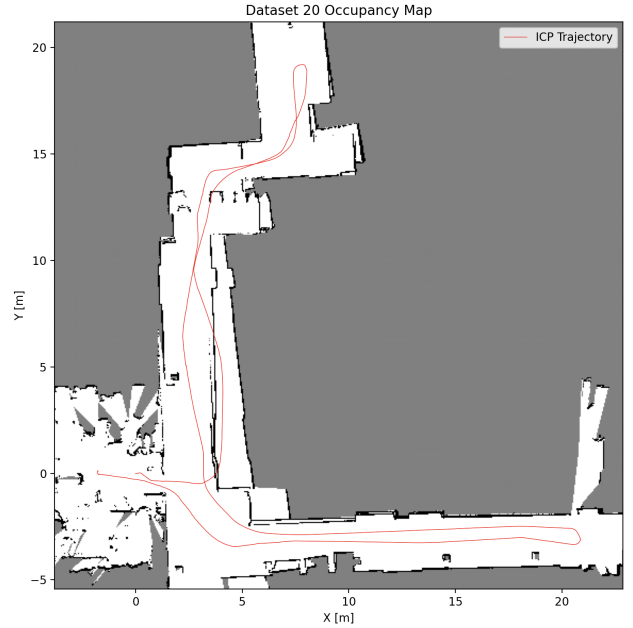


Fig. 7. Dataset 20 occupancy grid map constructed using the ICP-refined trajectory (red overlay).

a) *Observations.:* The final occupancy maps in Figs. 7–8 show sharp corridor walls and consistent room boundaries, suggesting that the LiDAR-to-body and body-to-world transformations are correct and that scan alignment is locally stable. In Dataset 20, the long lower corridor appears as a single coherent structure with minimal wall duplication, which would not occur if scan matching produced inconsistent relative transforms. In Dataset 21, the dominant corridor geometry is also clear; some areas show mild wall thickening and ghosting, consistent with residual drift over long trajectories. Overall, the occupancy maps qualitatively demonstrate that ICP refinement improves mapping quality compared to odometry-only integration.



Fig. 8. Dataset 21 occupancy grid map constructed using the ICP-refined trajectory (red overlay).

F. Texture Mapping

RGBD measurements from the Kinect are used to generate a textured floor map aligned with the 2-D occupancy grid. For each Kinect frame, depth is computed from disparity using

$$z = \frac{1.03}{-0.00304d + 3.31}. \quad (14)$$

Given pixel coordinates (u, v) and depth z , points are back-projected to the camera frame using the intrinsics

$$\mathbf{p}^c = z\mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \quad (15)$$

then transformed to the robot body frame and finally into the world frame using the SLAM pose at the closest timestamp:

$$\mathbf{p}^w = {}^w\mathbf{T}_b(t) {}^b\mathbf{T}_c \mathbf{p}^c. \quad (16)$$

We retain only points near the ground by thresholding height in the body/world frame (a narrow band around the floor plane), and rasterize the remaining points into a 2-D color grid with the same resolution and bounds as the occupancy grid. If multiple points land in the same cell, we overwrite or average colors (implementation choice); in practice, we update with the most recent observation to progressively “paint” the floor as the robot moves.

a) RGB-depth alignment and timing.: Because RGB and depth measurements do not share the same optical center, the

corresponding RGB pixel $(\text{rgb}i, \text{rgb}j)$ for a depth pixel (i, j) is computed using the provided calibration:

$$d_d = (-0.00304d + 3.31), \quad (17)$$

$$\text{depth} = \frac{1.03}{d_d}, \quad (18)$$

$$\text{rgb}i = \frac{526.37i + 19276 - 7877.07d_d}{585.051}, \quad (19)$$

$$\text{rgb}j = \frac{526.37j + 16662}{585.051}. \quad (20)$$

Kinect timestamps are irregular (dropped frames), so we associate each RGBD frame with the nearest SLAM pose in time.

1) Textured map progression and final results: To satisfy the requirement of visualizing textured maps over time, Fig. 9 and Fig. 10 show intermediate textured maps at several completion percentages for Datasets 20 and 21. The texture mapping implementation used to generate these figures is provided in `color_map.py`.

a) Observations.: The progression figures illustrate that early in the run (e.g., Dataset 20 at 10%), only a small region near the start is textured because few RGBD frames have been integrated and the robot has explored limited area. By mid-run (Dataset 20 at 50%, Dataset 21 at 65%), texture coverage expands along the visited corridors and becomes more continuous in areas where the pose estimates remain locally consistent. The final panels in Fig. 9 and Fig. 10 show coherent floor texture aligned with the corridor layout, indicating that camera projection and world-frame composition are implemented correctly. Remaining artifacts (holes, streaking, or duplicated bands) are most noticeable in long corridors and at turns; these effects are consistent with (i) missing or invalid disparity leading to dropped floor points and (ii) residual pose drift causing slight misprojection over distance. Overall, the textured maps become more interpretable as more frames are fused and provide a complementary visualization of free space alongside the occupancy grid.

G. Pose Graph Optimization Results

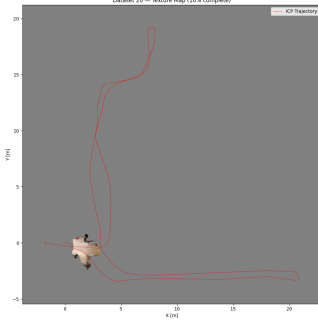
To assess the impact of pose graph optimization (PGO), we compare trajectories before and after optimization and examine textured maps reconstructed using optimized poses. The factor graph is optimized with nonlinear least squares in GTSAM using odometry/ICP factors and loop closure constraints.

Figures 11 and 12 illustrate how pose graph optimization improves trajectory consistency and overall map quality.

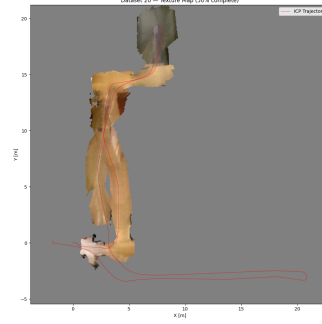
1) Trajectory Before vs After Optimization:

a) Observations: Pose graph optimization noticeably improves global consistency. The pre-optimization ICP trajectory shows drift, particularly along the long horizontal corridor and around the loop segment. After optimization, the trajectory becomes more geometrically consistent and better aligned with the expected corridor layout.

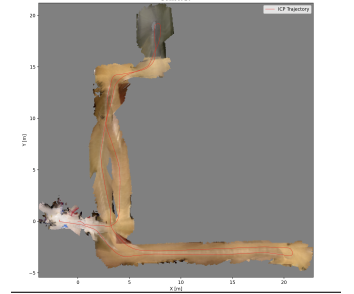
In Dataset 20, loop closure constraints reduce the separation between repeated passes through the same area and remove large-scale distortions present in the ICP-only trajectory. The



(a) 10% complete

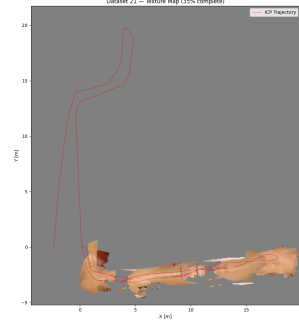


(b) 50% complete

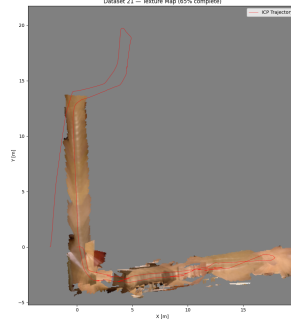


(c) Final

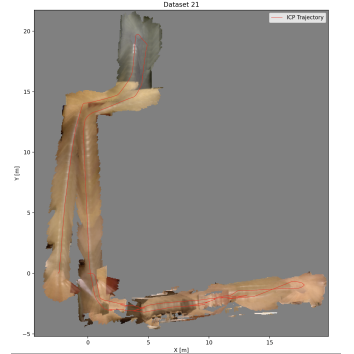
Fig. 9. Dataset 20 textured map over time. The map becomes denser and more coherent as more RGBD frames are integrated.



(a) 35% complete

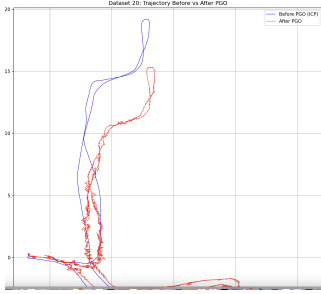


(b) 65% complete

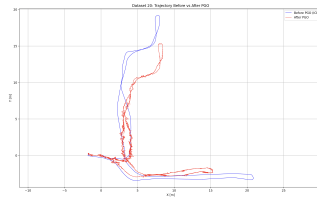


(c) Final

Fig. 10. Dataset 21 textured map over time. Long corridor regions gradually accumulate texture; residual drift can introduce mild smearing.



(a) Dataset 20



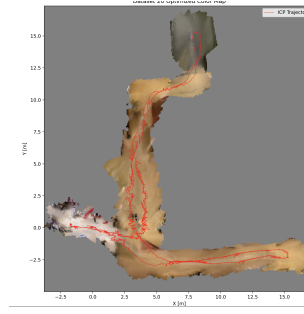
(b) Dataset 20 (alternate view)

Fig. 11. Trajectory comparison before and after pose graph optimization. Blue: ICP trajectory before optimization. Red: optimized trajectory.

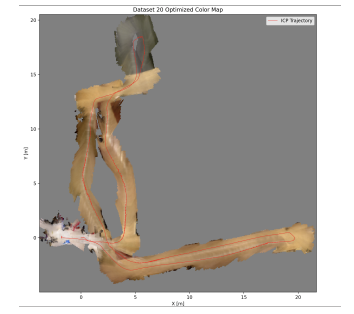
optimized solution follows a smoother and more physically plausible path through the environment.

Some small, high-frequency variations remain due to sensor noise and imperfect scan matching, but the dominant large-scale structure is substantially improved.

2) *Optimized Textured Maps:* After optimization, we reconstruct textured floor maps using the optimized poses. The mapping procedure is the same as in Sec. III-F, but the world-frame projection uses poses from the factor graph solution. The optimized maps are generated using `pose_map.py`, which projects RGBD measurements into the world frame with



(a) Dataset 20 optimized map



(b) Dataset 20 optimized map (alternate view)

Fig. 12. Optimized textured maps constructed using pose-graph optimized trajectories.

optimized trajectories.

a) *Observations:* The optimized textured maps are more spatially consistent than maps generated from ICP poses alone. Corridor textures align more cleanly with occupancy boundaries, and overlapping or duplicated texture regions are reduced.

In Dataset 20, the long corridor becomes more uniformly textured, indicating that pose graph optimization reduces accumulated pose error. The vertical corridor region also appears

more coherent, with fewer duplicated texture bands.

Some residual blur remains due to Kinect depth noise and imperfect temporal alignment between RGBD frames and robot poses. Nevertheless, the optimized results show that PGO substantially improves both trajectory accuracy and map quality, producing the most consistent representation of the environment in this pipeline.

IV. CONCLUSION

We implemented a LiDAR-based SLAM system for a differential-drive robot that combines encoder+IMU odometry, ICP scan matching, occupancy-grid mapping, RGBD texture mapping, and pose-graph optimization. Encoder and IMU measurements provide a stable motion prior, while scan matching reduces local drift and improves map structure. Pose graph optimization further improves global consistency by incorporating loop closures. Overall, the resulting system produces coherent trajectories and detailed occupancy and textured maps of the environment.

REFERENCES

- [1] F. Dellaert, "GTSAM: Georgia Tech Smoothing And Mapping," 2012.
- [2] P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992.